Guia de Estudo – Software Python (SuperSenha, Projeto 8)

Arquivo estudado: `supersenha.py`. Feito para a avaliação individual: o professor faz até 4 perguntas (1ª certa = 9, 2ª = 7, 3ª = 5, 4ª = 3). Então **domine primeiro as perguntas do Nível 1**, que são as mais prováveis de abrir a conversa.

---

1. Resumo do projeto em 30 segundos (decore)

> "Uma janela em Tkinter tem 4 campos onde o usuário digita 0 ou 1, um para cada chave do DIP switch. Ao clicar em **Tentar**, o Python junta os 4 dígitos numa string (ex.: `"1010"`) e envia pela porta serial (USB, 9600 baud) para o Arduino. O Arduino compara com a posição real das chaves DIP e responde `OK` ou `ERRO`. O Python fica checando a serial a cada 100 ms; quando chega a resposta, mostra **ACESSO LIBERADO!** (verde) ou **SENHA ERRADA!** (vermelho). O LCD I2C, ligado no Arduino, mostra a mesma informação."

Fluxo:

```
Usuário digita 0/1 → botão Tentar → enviarSenha() → ser.write("1010\n") → ARDUINO
                                                                              │ compara
com DIP
Tela mostra resultado ← mostrarResultado() ← verificarLeituraSerial() ← "OK"/"ERRO"
                                                                              └→ LCD
I2C
```

---

2. Mapa do código (onde está cada coisa)

Linhas	O que é
1–4	Imports: `tkinter` (interface), `PIL` (imagem/logo), `time` (espera), `serial` (pyserial, comunicação USB)
7–9	Configuração: `usarArduino`, `portaArduino`, `senhaTeste`
12	`respostaArduino`: variável global que guarda a última resposta
15–19	`mostrarResultado()`: pinta o texto na tela
22–28	`verificarLeituraSerial()`: lê a serial periodicamente
31–38	`limitarDigito()`: aceita só 0/1 e pula para o próximo campo
41–62	`enviarSenha()`: valida, monta a senha e envia (ou simula)
65–70	Abre a conexão serial (ou entra em modo de teste)
74–78	Criação e configuração da janela
81–90	Logo da UNIARA
93–139	Widgets: Label "Senha:", 4 `Entry`, botão, Label de resultado

Linhas	O que é
127–131	`bind` dos campos (evento de tecla)
141–144	Agenda a leitura serial e inicia o `mainloop()`

3. Perguntas prováveis, com resposta

NÍVEL 1 – Básicas (valem 9 pontos se acertar de primeira)

1. O que esse programa Python faz? É a interface gráfica. Coleta os 4 dígitos da senha, envia ao Arduino pela serial e exibe se o acesso foi liberado ou se a senha estava errada.

2. Qual biblioteca cria a janela? É nativa do Python? `tkinter`. É nativa, não precisa instalar. `PIL` (pillow) e `serial` (pyserial) precisam de `pip install`.

3. Para que serve a biblioteca `serial` (pyserial)? Para o Python conversar com o Arduino pela porta USB/serial (COM). Ela dá `Serial()`, `write()`, `readline()`, `in_waiting`.

4. Para que serve o `PIL` / `Pillow`? Para abrir e redimensionar a imagem da logo (`Image.open`, `resize`) e converter para um formato que o Tkinter entende (`ImageTk.PhotoImage`). O Tkinter sozinho não redimensiona PNG facilmente.

5. O que é `janela.mainloop()` e por que fica no final? É o loop de eventos do Tkinter: mantém a janela aberta, desenha a tela e reage a cliques/teclas. Tudo o que vem depois dele só executa quando a janela fecha, então ele deve ser a última linha.

6. O que faz o botão "Tentar"? Tem `command = enviarSenha`, então ao clicar chama a função `enviarSenha()`. Repare que é **sem parênteses**: passamos a função, não chamamos ela.

7. Qual a velocidade (baud rate) da comunicação? Precisa ser igual no Arduino? 9600. Sim, o `Serial.begin(9600)` do Arduino precisa ter exatamente o mesmo valor, senão chegam caracteres lixo.

8. O que é a porta `"com6"`? Como descobrir a sua? É a porta serial virtual que o Windows cria para o Arduino. Descobre-se no Arduino IDE em *Ferramentas → Porta* ou no Gerenciador de Dispositivos. Pode mudar se trocar a porta USB.

9. O que significa `usarArduino = False`? Modo de teste: o programa roda sem Arduino, simulando a resposta no próprio Python, comparando com `senhaTeste = "1010"`. Com `True`, abre a serial de verdade.

10. O que as 4 caixinhas representam? Uma para cada chave do DIP switch (1 a 4). `1` = chave ON, `0` = chave OFF.

NÍVEL 2 – Intermediárias (valem 7 pontos)

11. Explique a função `enviarSenha()` passo a passo. 1. Lê os 4 campos com `.get()` e coloca numa lista. 2. Percorre a lista; se algum campo não for "0" nem "1", mostra "Digite 0 ou 1 em cada campo" e dá `return` (sai sem enviar). 3. Concatena os 4 dígitos numa string (`"1010"`). 4. Se `usarArduino`, envia por `ser.write` e mostra "Verificando..."; senão compara com `senhaTeste` e simula a resposta.

12. Por que mandar `senha + "\n"`? O `\n` marca o **fim da mensagem**. O Arduino lê até encontrar o `\n` (por exemplo com `Serial.readStringUntil('\n')`). Sem ele, o Arduino não saberia onde a senha termina.

13. Por que `bytes(..., 'UTF-8')`? A serial trafega **bytes**, não texto. `write()` não aceita `str`. A conversão para bytes é obrigatória. Equivalente: `senha.encode()`.

14. Para que serve `ser.in_waiting`? Retorna quantos bytes já chegaram e estão esperando no buffer. O `if ser.in_waiting > 0` garante que só lemos quando o Arduino realmente mandou algo.

15. O que fazem `.decode('UTF-8')` e `.strip()` na linha 25? `readline()` devolve bytes, tipo `b'OK\r\n'`. O `decode` transforma em texto e o `strip` remove o `\r\n` do final. Sem o `strip`, `"OK\r\n" == "OK"` daria `False` e sempre mostraria senha errada.

16. Por que existe o `time.sleep(2.5)` depois de abrir a serial? Ao abrir a porta serial, o Arduino UNO **reinicia sozinho** (sinal DTR). Se enviarmos algo antes dele terminar de reiniciar, a mensagem se perde. Então esperamos.

17. O que é `janela.after(100, verificarLeituraSerial)` e por que a função chama a si mesma? `after` agenda uma função para rodar daqui a 100 ms sem travar a janela. Como a função se reagenda no final, vira uma checagem periódica (polling) enquanto o programa estiver aberto.

18. Por que não usar `while True` para ler a serial? O `while True` ocuparia o programa inteiro e o `mainloop()` nunca teria chance de redesenhar: a janela **congelaria**. Com `after` o controle volta ao Tkinter entre uma checagem e outra.

19. Para que serve `global respostaArduino`? Dentro de uma função, atribuir a uma variável cria uma variável **local**. O `global` diz que queremos alterar a variável de fora da função. Sem ele, a atribuição não afetaria a variável global.

20. Como funciona a função `limitarDigito`? É chamada a cada tecla solta. Pega o texto; se tem mais de 1 caractere, fica só com o último; apaga o campo; se o caractere for "0" ou "1", reinsere e move o foco (`focus()`) para o próximo campo. Qualquer outra tecla (letra, número 2–9) deixa o campo vazio.

21. O que é `bind("<KeyRelease>", ...)`? E o `lambda e:`? `bind` liga um **evento** a uma função: ao soltar uma tecla naquele campo, executa a função. O Tkinter sempre passa um objeto evento (`e`) para quem for chamado por `bind`. O `lambda e:` recebe esse evento (não usamos) e chama `limitarDigito` com os argumentos que queremos.

22. Por que usar `lambda` e não passar `limitarDigito` direto? Porque `limitarDigito` precisa de 2 argumentos (campo atual e próximo), mas o `bind` só entrega o evento. O `lambda` faz a ponte.

23. Quando digito no 4º campo, para onde vai o foco? Para o `botaoTentar`. Depois disso, **Espaço** (ou Enter, dependendo do sistema) aciona o botão.

NÍVEL 3 – Mais profundas (valem 5 pontos)

24. Qual a diferença entre `place`, `pack` e `grid`? Gerenciadores de layout. `place` posiciona por coordenadas exatas em pixels (x, y), é o que usamos. `pack` empilha os widgets automaticamente. `grid` organiza em linhas e colunas.

25. Por que a linha `labelUniara.image = figuraUniara`? O Tkinter **não guarda** referência da imagem. Se a variável sair de escopo, o Python (garbage collector) apaga a imagem e a Label aparece vazia. Guardar a referência no próprio Label evita isso.

26. O que faz `janela.iconphoto(False, figuraUniara)`? Troca o ícone da janela (aquela penazinha do Tk) pela logo da UNIARA. O `False` significa que vale só para esta janela, não para as janelas futuras.

27. O que faz `janela.resizable(False, False)`? Impede de redimensionar a janela na largura e na altura. Assim o layout em coordenadas fixas (`place`) nunca desalinha.

28. Como o Python sabe se é para mostrar verde ou vermelho? Em `mostrarResultado()`: se `respostaArduino == "OK"` mostra verde. **Qualquer outra resposta** (ERRO, lixo, vazio) cai no `else` e mostra vermelho "SENHA ERRADA!".

29. Quem decide se a senha está certa: o Python ou o Arduino? No modo real, o **Arduino**. Ele lê as chaves DIP físicas e compara. O Python só envia a tentativa e exibe a resposta. No modo teste, o Python simula essa decisão com `senhaTeste`.

30. O que acontece se o Arduino não estiver conectado e `usarArduino = True`? `serial.Serial(...)` lança `SerialException` (porta não encontrada) e o programa fecha com erro antes de abrir a janela. Por isso existe o modo de teste.

31. Se eu trocar de `com6` para outra porta, o que muda? Só a variável `portaArduino`. Por isso ela fica no bloco de configuração no topo.

32. Por que o Python importa `serial` mesmo no modo de teste? O `import` fica no topo e roda sempre. Então, mesmo no modo de teste, o pyserial precisa estar instalado, senão dá `ModuleNotFoundError`. Instala-se com `py -m pip install pyserial`.

33. Por que existe o texto "Verificando..."? Feedback visual enquanto o Arduino não responde. Quando chega a resposta, `mostrarResultado()` o substitui.

34. Qual a diferença entre `Entry` e `Label`? `Entry` é um campo de **entrada**: o usuário digita e lemos com `.get()`. `Label` é só **texto exibido**; mudamos com `.config(text=...)`.

35. O que faz `.config(text=..., fg=...)`? Altera propriedades de um widget depois de criado. `text` muda o texto, `fg` (foreground) a cor da letra, `bg` a cor de fundo.

NÍVEL 4 – Pegadinhas / "e se..." (valem 3 pontos)

36. E se o usuário digitar "2" ou uma letra? `limitarDigito` apaga o campo e não avança o foco. Se ainda assim chegar vazio ao botão, `enviarSenha` mostra "Digite 0 ou 1 em cada campo" e não envia nada.

37. E se o usuário deixar um campo em branco e clicar em Tentar? Mesma coisa: o `for` acha um campo diferente de "0" e "1", exibe o aviso e o `return` impede o envio.

38. E se o Arduino demorar ou nunca responder? A tela fica em "Verificando..." para sempre. O código não tem timeout. Seria uma melhoria: contar o tempo e mostrar "Sem resposta do Arduino".

39. E se o Arduino mandar a resposta sem o `\n`? `readline()` espera o `\n`. Como o `Serial()` foi criado **sem** `timeout`, a leitura pode ficar bloqueada e travar a janela. Por isso o Arduino deve usar `Serial.println` (que manda o `\n`) e não `Serial.print`.

40. Por que 100 ms? E se fosse 1 ms, ou 5 segundos? 100 ms é um bom equilíbrio: a resposta parece instantânea ao usuário e não sobrecarrega o processador. Com 1 ms gastaríamos CPU à toa; com 5 s a tela demoraria até 5 s para reagir.

41. A senha é segura? Não como sistema real: trafega em texto puro pela serial e a senha é a posição do DIP, que qualquer um vê. É um projeto didático de interface, sem criptografia.

42. Dá para trocar o DIP por 8 chaves? Sim: mais campos de entrada no Python (e mais pinos no Arduino), e a string teria 8 dígitos. O protocolo não muda.

43. Por que o Python envia a senha e não o Arduino envia o estado do DIP para o Python comparar? O enunciado pede que o **display LCD** informe o resultado, então o Arduino precisa conhecer o resultado e decidir. Se só o Python soubesse, o Arduino teria que receber a resposta de volta.

44. O que acontece no `ser.write` se a porta estiver aberta em outro programa (ex.: Monitor Serial do Arduino IDE)? A porta só aceita **um programa por vez**. O Python dá erro de acesso negado ao abrir. Fechar o Monitor Serial resolve.

45. Como você testaria o Python sem ter o Arduino? Colocando `usarArduino = False`. O programa simula a resposta usando `senhaTeste`. Também dá para testar a serial com um loopback ou um simulador de porta virtual.

---

4. Exercícios de "mexer no código" (o professor pode pedir ao vivo)

1. **Trocar a senha de teste**: alterar `senhaTeste = "1010"` para outro valor (linha 9).
2. **Mudar a cor de fundo**: alterar `corFundo` (linha 72).
3. **Mudar a mensagem**: editar os textos em `mostrarResultado()` (linhas 17 e 19).
4. **Mudar a porta**: `portaArduino = "com3"` (linha 8).
5. **Mudar o tamanho da janela**: `geometry('700x400')` (linha 75).
6. **Mover a logo ou o botão**: valores `x` e `y` do `place`.
7. **Trocar o baud rate**: linha 66 (e no Arduino também!).
8. **Adicionar um 5º dígito**: criar `digito5`, incluí-lo na lista e na concatenação, e ajustar o `bind`.

---

5. Ponte com o Arduino (para não travar se a pergunta cruzar áreas)

- **Protocolo usado**: Python → Arduino: `"1010\n"` (4 caracteres + quebra de linha). Arduino → Python: `"OK\n"` ou `"ERRO\n"`.
- **DIP com `INPUT_PULLUP`** (conforme o guia de montagem): o pino fica em HIGH sozinho; com a chave ON ele liga ao GND e vira **LOW**. Ou seja, o Arduino precisa **inverter** a leitura: LOW = 1 (ON).
- **Ligações**: DIP nos pinos D2–D5 e GND; LCD I2C em 5V, GND, SDA = A4, SCL = A5.
- **I2C** usa só 2 fios (SDA = dados, SCL = clock) e cada dispositivo tem um endereço (comum: `0x27` ou `0x3F`).
- O mesmo `Serial.begin(9600)` precisa existir no Arduino.

---

6. Dicas para a apresentação

- Se não souber algo, responda o que sabe e **raciocine em voz alta**; o professor valoriza entender a lógica.
- Tenha o programa **rodando** no dia: nota depende do projeto funcionar. Teste antes com `usarArduino = True`, na porta COM certa, com o Monitor Serial fechado.
- Os arquivos `logoUniara.png` e `supersenha.py` precisam estar **na mesma pasta**, senão o `Image.open` falha.
- Saiba apontar no código onde está cada parte do enunciado: 4 chaves DIP → 4 `Entry`; botão "Tentar" → `botaoTentar`; "acesso liberado / senha errada" → `mostrarResultado()`; logo da UNIARA → linhas 81–90.